

MLP, inizializzazione, attivazioni, momentum e dropout

Obiettivi formativi

Al termine dell'attività, studentesse e studenti dovranno essere in grado di:

- costruire un MLP per classificazione in Keras/TensorFlow con output softmax e loss di cross-entropy.
- confrontare inizializzazioni diverse, in particolare Glorot e He, in relazione alla funzione di attivazione utilizzata.
- usare ReLU, Leaky ReLU ed ELU e discuterne il comportamento in termini di stabilità del training e rischio di dying ReLU.
- analizzare l'effetto del momentum sulla convergenza della loss e della validation accuracy.
- usare il dropout come tecnica di regolarizzazione e valutarne l'effetto sull'overfitting.

Prerequisiti

Sono richieste conoscenze di base su reti neurali feedforward, discesa del gradiente, classificazione multiclasse, softmax e cross-entropy. È inoltre richiesto un uso di base di Python, NumPy, Matplotlib e TensorFlow/Keras.

Dataset

Per tutta l'attività si userà un dataset di classificazione multiclasse di immagini già pronto in Keras, preferibilmente Fashion-MNIST. Il dataset contiene immagini grayscale di dimensione 28×28 e un'etichetta di classe per ciascuna istanza, risultando adatto per osservare differenze di training senza introdurre architetture convoluzionali.

Struttura dell'attività

L'attività è suddivisa in cinque parti:

1. costruzione di un modello baseline;
2. confronto tra inizializzazione Glorot e He;
3. confronto tra ReLU, Leaky ReLU ed ELU;

4. confronto tra SGD senza momentum e SGD con momentum;
5. introduzione del dropout e analisi dell'overfitting.

Parte 1 – Modello baseline

Obiettivo

Costruire un MLP di riferimento e raccogliere le curve di training e validazione.

Istruzioni

1. Caricare Fashion-MNIST o MNIST.
2. Normalizzare i pixel dividendo per 255.
3. Suddividere il training set in training e validation set.
4. Definire un MLP con questa struttura di base:
 - Flatten()
 - Dense(300, activation='relu')
 - Dense(100, activation='relu')
 - Dense(10, activation='softmax')
5. Compilare il modello con:
 - ottimizzatore SGD(learning_rate=0.01)
 - loss sparse_categorical_crossentropy
 - metrica accuracy.
6. Addestrare per 20 epoche.
7. Rappresentare graficamente loss, val_loss, accuracy, val_accuracy.

Codice guida

```
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt

(X_train_full, y_train_full), (X_test, y_test) =
keras.datasets.fashion_mnist.load_data()
X_train_full = X_train_full / 255.0
X_test = X_test / 255.0
```

```

X_valid, X_train = X_train_full[:5000], X_train_full[5000:]
y_valid, y_train = y_train_full[:5000], y_train_full[5000:]

model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(300, activation="relu"),
    keras.layers.Dense(100, activation="relu"),
    keras.layers.Dense(10, activation="softmax")
])

model.compile(loss="sparse_categorical_crossentropy",
              optimizer=keras.optimizers.SGD(learning_rate=0.01),
              metrics=["accuracy"])

history = model.fit(X_train, y_train, epochs=20,
                    validation_data=(X_valid, y_valid))

```

Domande guida

- Dopo quante epoche il modello inizia eventualmente a overfittare?
- La validation accuracy segue da vicino la training accuracy oppure no?
- Le curve sono regolari o mostrano oscillazioni importanti?

Parte 2 – Glorot vs He

Obiettivo

Osservare l'effetto dell'inizializzazione dei pesi sul training di una rete con ReLU.

Istruzioni

Costruire due modelli identici al baseline, cambiando solo il `kernel_initializer` dei layer nascosti:

- Modello A: `kernel_initializer="glorot_uniform"`.
- Modello B: `kernel_initializer="he_normal"` oppure `"he_uniform"`.

Addestrare entrambi con gli stessi iperparametri del modello baseline.

Codice guida

```
model_glorot = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(300, activation="relu", kernel_initializer="glorot_uniform"),
    keras.layers.Dense(100, activation="relu", kernel_initializer="glorot_uniform"),
    keras.layers.Dense(10, activation="softmax")
])

model_he = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(300, activation="relu", kernel_initializer="he_normal"),
    keras.layers.Dense(100, activation="relu", kernel_initializer="he_normal"),
    keras.layers.Dense(10, activation="softmax")
])
```

Domande

- Quale inizializzazione sembra favorire una discesa più rapida della loss nelle prime epoche?
- Le differenze si osservano solo all'inizio o anche nelle metriche finali?
- Perché He è in genere una scelta naturale con ReLU e varianti di ReLU?

Parte 3 – ReLU, Leaky ReLU ed ELU

Obiettivo

Confrontare tre attivazioni molto usate e collegarle ai problemi di saturazione e dying ReLU.

Scelta consigliata delle varianti

Per un laboratorio breve, si consiglia di confrontare:

- ReLU, come baseline standard;
- Leaky ReLU, come variante semplice che lascia passare un piccolo gradiente anche per input negativi;
- ELU, come variante smooth con valori negativi che può migliorare la dinamica del training in alcuni casi.

Istruzioni

Definire tre modelli identici, usando inizializzazione He per tutti i layer nascosti e cambiando soltanto l'attivazione.

Modello con ReLU

```
model_relu = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(300, activation="relu", kernel_initializer="he_normal"),
    keras.layers.Dense(100, activation="relu", kernel_initializer="he_normal"),
    keras.layers.Dense(10, activation="softmax")
])
```

Modello con Leaky ReLU

```
model_lrelu = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(300, kernel_initializer="he_normal"),
    keras.layers.LeakyReLU(alpha=0.01),
    keras.layers.Dense(100, kernel_initializer="he_normal"),
    keras.layers.LeakyReLU(alpha=0.01),
    keras.layers.Dense(10, activation="softmax")
])
```

Modello con ELU

```
model_elu = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(300, activation="elu", kernel_initializer="he_normal"),
    keras.layers.Dense(100, activation="elu", kernel_initializer="he_normal"),
    keras.layers.Dense(10, activation="softmax")
])
```

Domande guida

- Quale attivazione porta alla validation accuracy migliore?
- Quale attivazione sembra più stabile nelle prime epoche?
- Il modello con Leaky ReLU mostra vantaggi rispetto a ReLU standard?
- In quali casi ELU sembra comportarsi meglio o peggio delle altre varianti?

Parte 4 – Momentum

Obiettivo

Capire come il momentum modifichi la dinamica dell'ottimizzazione con SGD.

Istruzioni

Prendere il modello che ha funzionato meglio nelle parti precedenti e addestrarlo in due configurazioni:

- `SGD(learning_rate=0.01, momentum=0.0);`
- `SGD(learning_rate=0.01, momentum=0.9).`

Confrontare le curve di loss e accuracy.

Codice guida

```
optimizer_plain = keras.optimizers.SGD(learning_rate=0.01, momentum=0.0)
optimizer_momentum = keras.optimizers.SGD(learning_rate=0.01, momentum=0.9)
```

Domande guida

- Il momentum accelera la convergenza?
- Le oscillazioni della loss diminuiscono o aumentano?
- A parità di epoche, il modello con momentum raggiunge una validation accuracy maggiore?
- Che relazione c'è tra osservazioni sperimentali e interpretazione fisica del momentum come media mobile degli aggiornamenti?

Parte 5 – Dropout

Obiettivo

Usare il dropout come regolarizzatore e valutarne l'effetto sull'overfitting.

Istruzioni

Prendere il modello migliore e aggiungere un layer Dropout dopo ciascun hidden layer. Ripetere l'esperimento con almeno due valori del tasso di dropout, ad esempio 0.3 e 0.5.

Codice guida

```
model_dropout = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
```

```

keras.layers.Dense(300, activation="relu", kernel_initializer="he_normal"),
keras.layers.Dropout(0.3),
keras.layers.Dense(100, activation="relu", kernel_initializer="he_normal"),
keras.layers.Dropout(0.3),
keras.layers.Dense(10, activation="softmax")
1)

```

Domande guida

- La training accuracy diminuisce con dropout? Perché questo non è necessariamente un male?
- La validation accuracy migliora rispetto al modello senza dropout?
- Quale valore di dropout sembra il più efficace sul dataset usato?
- Il dropout riduce la distanza tra training e validation performance?

Confronto finale

Completare una tabella di sintesi come la seguente:

Configurazione	Val accuracy finale	Val loss finale	Osservazioni
Glorot + ReLU + SGD			
He + ReLU + SGD			
He + Leaky ReLU + SGD			
He + ELU + SGD			
He + Leaky ReLU + Momentum			
He + Leaky ReLU + Momentum + Dropout			

Richieste per la consegna

La consegna deve includere:

- un notebook .ipynb ben organizzato, con una sezione per ogni parte dell'attività;
- i grafici di training/validation loss e accuracy per tutti gli esperimenti richiesti;
- una breve discussione finale di circa 10–15 righe che risponda a questa domanda:

Quale combinazione tra inizializzazione, funzione di attivazione, momentum e dropout ha prodotto i risultati migliori, e come si possono interpretare questi risultati alla luce della teoria presentata a lezione?

Criteri di valutazione suggeriti

Criterio	Peso
Corretta implementazione degli esperimenti	40%
Qualità dei grafici e chiarezza dell'analisi	25%
Capacità di collegare risultati e teoria	25%
Ordine, leggibilità e riproducibilità del notebook	10%

Estensioni facoltative

Per studenti o gruppi che terminano in anticipo, si possono proporre una o più estensioni:

- confronto con SELU e AlphaDropout, per collegarsi al tema delle self-normalizing neural networks.
- confronto tra SGD con momentum e Adam, discutendo vantaggi e limiti di ciascun ottimizzatore.
- analisi dell'effetto del learning rate in combinazione con momentum.

Sintesi conclusiva

Questa attività permette di osservare in modo concreto che la qualità del training di un MLP non dipende solo dall'architettura, ma anche da inizializzazione, attivazione, ottimizzazione e regolarizzazione. In particolare, il capitolo 11 insiste sul fatto che inizializzazioni adeguate, attivazioni non saturanti, momentum e dropout sono strumenti complementari per addestrare reti profonde in modo più stabile ed efficace.